INFORMATIONSBLATT

IB01 – INNER JOIN – Tabellen verbinden

SQL · LS07 – Tabellen verbinden (JOIN) · IB01 | 2BKI2 · Schuljahr 2026/2027

🎯 LERNZIEL

Nach diesem Informationsblatt können Sie …

- erklären, warum man nach der Normalisierung Tabellen wieder verbinden muss,
- zwei Tabellen mit INNER JOIN … ON über Primär- und Fremdschlüssel verbinden,
- Tabellen-Aliasse verwenden und mehrdeutige Spaltennamen vermeiden,
- drei und mehr Tabellen in einer Abfrage verbinden.

1 · Warum JOIN?

In *fahrrad* steht nur die TypNr, die Bezeichnung steht in *fahrradtyp*. Ein JOIN hängt an jede Zeile der einen Tabelle die **passende** Zeile der anderen – „passend“ heißt: Fremdschlüssel = Primärschlüssel.

fahrrad

FahrradNr	Farbe	TypNr
1	schwarz	1
2	rot	2
4	grau	3

ON f.TypNr = t.TypNr

fahrradtyp

TypNr	Bezeichnung
1	Citybike
2	Kinderrad
3	E-Bike
6	Tandem

kein Partner → fehlt beim INNER JOIN

Ergebnis

FahrradNr	Farbe	TypNr	Bezeichnung
1	schwarz	1	Citybike
2	rot	2	Kinderrad

… usw.

Abb. 1 – INNER JOIN: Fahrrad und Fahrradtyp werden über die TypNr zusammengeführt.

2 · Die Syntax

```
SELECT spalten
FROM tabelle1
  INNER JOIN tabelle2 ON tabelle1.fremdschlüssel = tabelle2.primärschlüssel;
```

```
SELECT fahrrad.FahrradNr, fahrrad.Farbe, fahrradtyp.Bezeichnung
FROM fahrrad
  INNER JOIN fahrradtyp ON fahrrad.TypNr = fahrradtyp.TypNr;
```

▶ ERGEBNIS

FahrradNr	Farbe	Bezeichnung
1	schwarz	Citybike
3	weiß	Citybike
7	grün	Citybike
15	grün	Citybike
17	rot	Citybike
2	rot	Kinderrad
… 12 weitere Zeilen …		

18 Zeilen

Die Bedingung hinter `ON` ist fast immer „Fremdschlüssel = Primärschlüssel“ – genau der Pfeil aus dem Relationenmodell: **TypNr** → fahrradtyp.

3 · Tabellen-Aliasse und mehrdeutige Spalten

Die Spalte `TypNr` gibt es in beiden Tabellen. Wer sie ohne Tabellennamen verwendet, bekommt:

```
SELECT FahrradNr, TypNr, Bezeichnung
FROM fahrrad
  INNER JOIN fahrradtyp ON fahrrad.TypNr = fahrradtyp.TypNr;
```

Fehler `#1052 - Feld 'TypNr' in field list ist nicht eindeutig`

Mit **Aliassen** (Kurznamen für Tabellen) wird der Befehl kürzer und eindeutig:

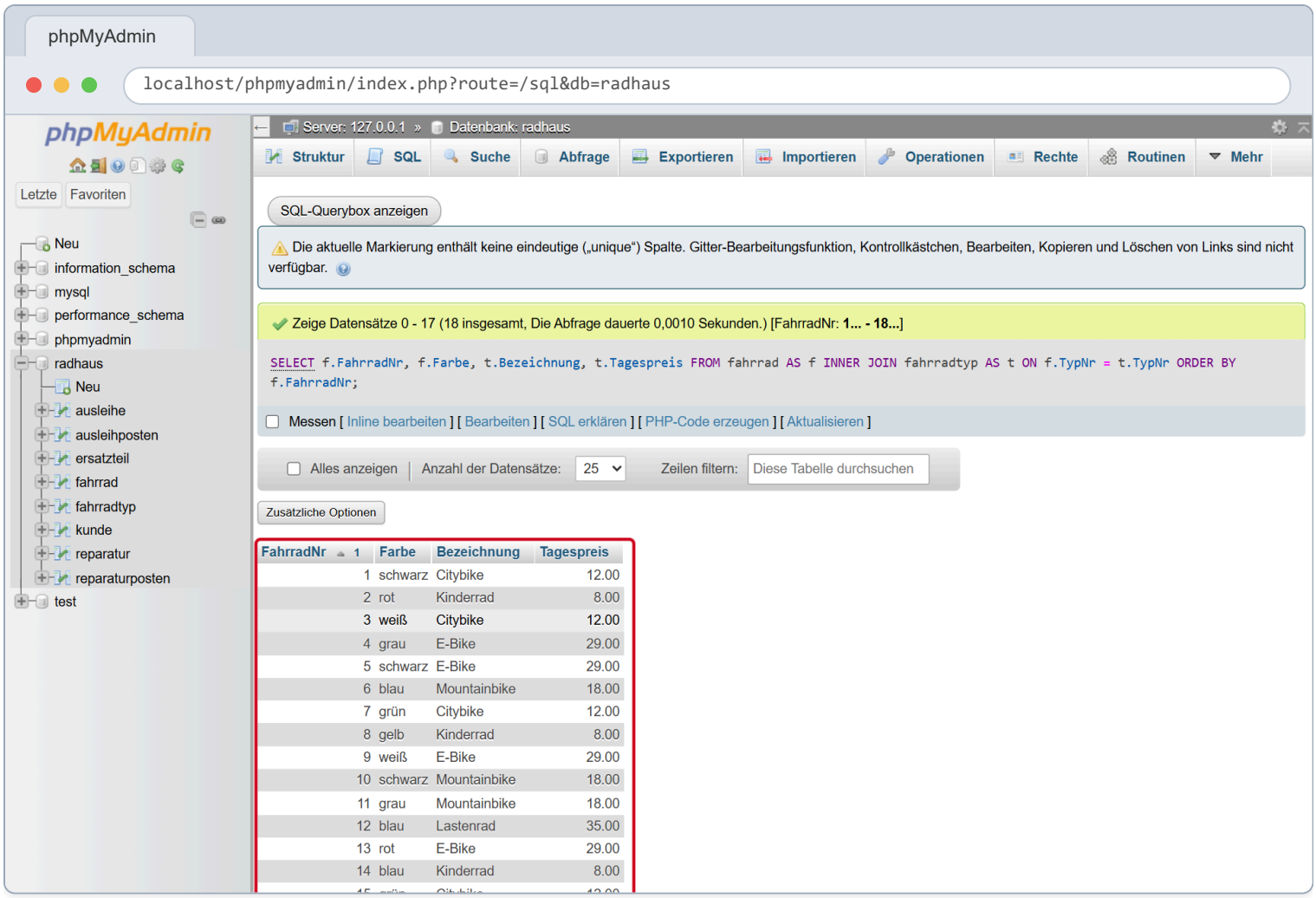


Abb. 2 – INNER JOIN mit den Aliassen f und t in phpMyAdmin.

> ✅ **SCHREIBWEISE**
>
> `fahrrad AS f` – danach heißt die Tabelle im ganzen Befehl nur noch `f`. Schreiben Sie bei JOINs **jede** Spalte mit Alias: `f.Farbe`, `t.Bezeichnung`. Das Wort `INNER` darf fehlen – `JOIN` allein bedeutet INNER JOIN.

4 · Mehr als zwei Tabellen

Welche Kunden haben welche Fahrradtypen gemietet? Der Weg führt durch fünf Tabellen: *kunde* → *ausleihe* → *ausleihposten* → *fahrrad* → *fahrradtyp*. Für jeden Schritt ein JOIN:

```
SELECT a.Datum, k.Nachname, f.FahrradNr, t.Bezeichnung
FROM kunde AS k
  INNER JOIN ausleihe      AS a  ON k.KundenNr  = a.KundenNr
  INNER JOIN ausleihposten AS ap ON a.AusleihNr = ap.AusleihNr
  INNER JOIN fahrrad       AS f  ON ap.FahrradNr = f.FahrradNr
  INNER JOIN fahrradtyp    AS t  ON f.TypNr     = t.TypNr
WHERE a.Datum BETWEEN '2026-05-01' AND '2026-05-15'
ORDER BY a.Datum;
```

▶ **ERGEBNIS**

Datum	Nachname	FahrradNr	Bezeichnung
2026-05-01	Fischer	1	Citybike
2026-05-01	Fischer	2	Kinderrad
2026-05-01	Fischer	8	Kinderrad
2026-05-02	Braun	9	E-Bike
2026-05-09	Weber	12	Lastenrad
2026-05-14	Becker	3	Citybike
2026-05-14	Becker	14	Kinderrad

7 Zeilen

> 💡 **FAUSTREGEL**
>
> Bei *n* Tabellen braucht man *n − 1* JOIN-Bedingungen. WHERE, ORDER BY usw. folgen wie gewohnt nach den JOINs.

> ⚠️ **ON VERGESSEN?**
>
> Fehlt die Verbindungsbedingung, verbindet MariaDB *jede* Zeile mit *jeder*: 18 Fahrräder × 6 Typen = 108 Zeilen Unsinn (*kartesisches Produkt*).
